OpenClaw Multi-Agent System

OpenClaw Multi-Agent System

- 1. Course Content
  - Course Overview
  - Learning Objectives
- 2. Case Demonstration
  - 2.1 Creating an Agent Using CLI Agent Wizard
  - 2.2 Agent Directory Structure
  - 2.3 Verifying Agent Creation
  - 2.4 Restarting Gateway to Apply Configuration
  - 2.5 Agent Conversation
- 3. Deleting an Agent
  - 3.1 Deleting Using CLI Command
  - 3.2 Manually Cleaning Up Agent Directory
  - 3.3 Cleaning Up Binding Configuration
- 4. CLI Commands and Parameter Description

1. Course Content

Course Overview

- The OpenClaw multi-agent system allows running multiple **isolated agents** within the same Gateway, each with its own workspace, state directory, and session history
- The multi-agent architecture is suitable for multi-role collaboration scenarios, for example, one agent handles programming development while another handles social media operations, with complete data isolation and no interference between them

Learning Objectives

1. Understand the concept and applicable scenarios of the OpenClaw multi-agent system
2. Learn how to create new agents using CLI commands
3. Learn how to edit agent prompt files (AGENTS.md / SOUL.md / IDENTITY.md, etc.)
4. Learn how to delete agents

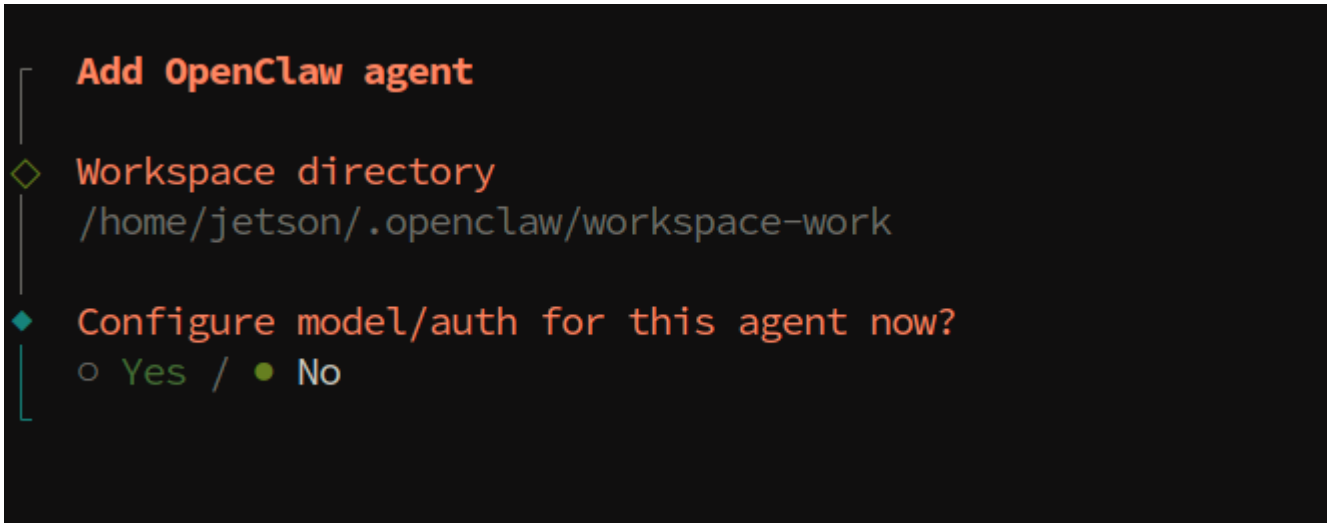
2. Case Demonstration

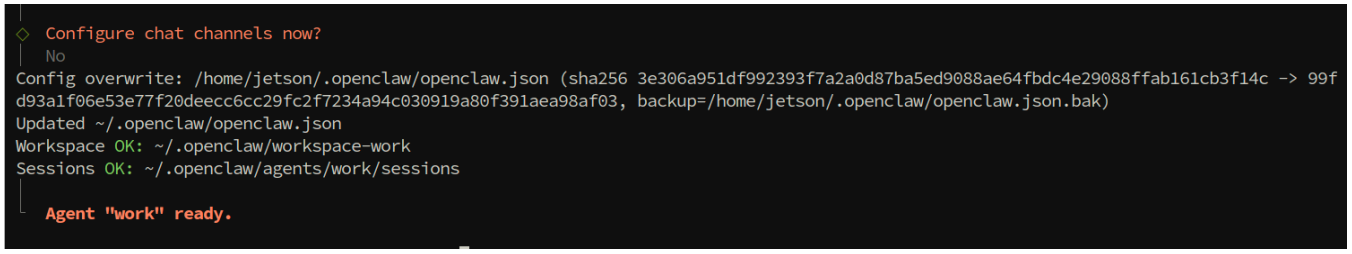
OpenClaw runs a default agent (agentId is `main`). The following methods can be used to create additional isolated agents.

2.1 Creating an Agent Using CLI Agent Wizard

For example, to create a work agent named `work`:

```
openclaw agents add work
```





After execution, OpenClaw automatically performs the following:

1. Creates the agent directory: `~/.openclaw/agents/<agentId>/`
2. Creates workspace bootstrap files: including `AGENTS.md`, `SOUL.md`, `IDENTITY.md`, `TOOLS.md`, `USER.md`, etc.

2.2 Agent Directory Structure

After creating an agent, a complete file structure is generated under the `~/.openclaw/agents/<agentId>/` directory:

```
~/.openclaw/agents/<agentId>/
├── agent/
│   ├── auth-profiles.json    # Authentication configuration file
│   ├── config.json           # Agent configuration
│   └── ...
└── sessions/                 # Session history storage
    └── <SessionId>.jsonl
```

The agent's workspace files are located in their respective workspace directories, where you can edit bootstrap files to define the agent's behavior and personality.

2.3 Verifying Agent Creation

After creation, you can view all agents and their binding status using the following command:

```
openclaw agents list --bindings
```

```
Agents:
- main (default)
  Workspace: ~/.openclaw/workspace
  Agent dir: ~/.openclaw/agents/main/agent
  Model: bailian/qwen3.6-plus
  Routing rules: 1
  Routing: Feishu default
  Providers:
    - Feishu default: configured
  Routing rules:
    - feishu accountId=default
- work
  Workspace: ~/.openclaw/workspace-work
  Agent dir: ~/.openclaw/agents/work/agent
  Model: bailian/qwen3.6-plus
  Routing rules: 0
Routing rules map channel/account/peer to an agent. Use --bindings for full rules.
Channel status reflects local config/creds. For live health: openclaw channels status --probe.
```

2.4 Restarting Gateway to Apply Configuration

After creating an agent or modifying binding configuration, you need to restart the Gateway:

```
openclaw gateway restart
```

After restart, verify the agent and channel status:

```
openclaw agents list --bindings
```

```
yahboom@yahboom:~$ openclaw agents list --bindings

🦞 OpenClaw 2026.3.13 (61d171a) — Welcome to the command line: where dreams compile and confidence segfaults.

Agents:
- main (default)
  Workspace: ~/.openclaw/workspace
  Agent dir: ~/.openclaw/agents/main/agent
  Model: bailian/qwen3.6-plus
  Routing rules: 0
  Routing: default (no explicit rules)
- work
  Workspace: ~/.openclaw/workspace-work
  Agent dir: ~/.openclaw/agents/work/agent
  Model: bailian/qwen3.6-plus
  Routing rules: 0
Routing rules map channel/account/peer to an agent. Use --bindings for full rules.
Channel status reflects local config/creds. For live health: openclaw channels status --probe.
```

2.5 Agent Conversation

```
openclaw tui
```

- In the dialog box, enter **/agent**, press Enter, and select the newly created agent

```
session agent:main:tui-ed9165f2-9747-401a-a003-4ec49571d5c9

new session: tui-ed9165f2-9747-401a-a003-4ec49571d5c9
connected | idle
agent main | session tui-ed9165f2-9747-401a-a003-4ec49571d5c9 | bailian/qwen3.6-plus | tokens ?/1.0m

/agent

→ agent                    Switch agent (or open picker)
  agents                   Open agent picker
  subagents                List, kill, log, spawn, or steer subagent runs for this session.
```

```
session agent:main:tui-ed9165f2-9747-401a-a003-4ec49571d5c9

new session: tui-ed9165f2-9747-401a-a003-4ec49571d5c9
connected | idle
agent main | session tui-ed9165f2-9747-401a-a003-4ec49571d5c9 | bailian/qwen3.6-plus | tokens ?/1.0m

search: >

  main  default
→ work (work)
```

- Enter text in the input box to converse with the agent

```
jetson@yahboom: ~
openclaw tui - ws://127.0.0.1:18789 - agent work (work) - session main

session agent:work:main

hello

Hey. I just came online — fresh workspace, no memories yet, not even a name.

So... who are you? And who do you want me to be?

I need a few things to get started:
- A name — something you'd actually want to say out loud
- A vibe — formal butler? sarcastic sidekick? chill and competent?
- An emoji — my signature

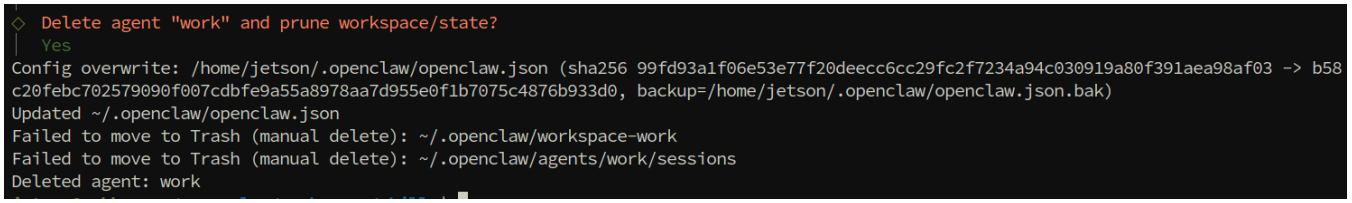
I've got suggestions if you're stuck, or you can just tell me who I am. Also curious what I'm working
with here — I see robot arm and camera tools in my kit, which is... not the usual onboarding.
connected | idle
agent work (work) | session main (openclaw-tui) | bailian/qwen3.8-max | tokens 29k/1.0m (3%)
```

3. Deleting an Agent

3.1 Deleting Using CLI Command

For example, to delete an agent named `work`:

```
openclaw agents delete work
```



3.2 Manually Cleaning Up Agent Directory

After executing the CLI command, it is recommended to also clean up the following residual data:

```
# Delete agent data directory
rm -rf ~/.openclaw/agents/work

# Delete agent workspace
rm -rf ~/.openclaw/workspace-work
```

3.3 Cleaning Up Binding Configuration

After deleting an agent, you also need to clear the corresponding binding records and agent list entries from the `openclaw.json` configuration file:

1. Open the configuration file: `~/.openclaw/openclaw.json`
2. Remove the corresponding agent entry from `agents.list`
3. Remove binding rules related to this agent from `bindings`
4. Save and restart the Gateway:

```
openclaw gateway restart
```

4. CLI Commands and Parameter Description

Agent Management Commands:

- `openclaw agents add <agentId>`: Creates a new isolated agent. `agentId` is the unique identifier of the agent (such as `work`, `dev`). After execution, the agent directory and workspace bootstrap files are automatically created
- `openclaw agents list --bindings`: Views all agents and their channel binding status. The `--bindings` parameter displays the interaction channel information bound to each agent
- `openclaw agents delete <agentId>`: Deletes the specified agent. After execution, it is recommended to manually clean up the residual data directory and workspace

- `openclaw tui`: Launches the terminal interactive interface. Enter `/agent` to switch the agent for the current conversation

Workspace Bootstrap File Description:

- `AGENTS.md`: The agent's main bootstrap file, defining the agent's core behavior rules, tool usage strategies, and task execution flow
- `SOUL.md`: The agent's soul file, defining the agent's personality traits and interaction style
- `IDENTITY.md`: The agent's identity file, defining identity information such as the agent's name and role positioning
- `TOOLS.md`: The tool configuration file, defining the list of MCP Tools available to the agent and usage constraints
- `USER.md`: The user information file, recording user preferences and background information to help the agent better serve the user

Configuration File Parameter Description:

- `agents.list`: Located in `~/.openclaw/openclaw.json`, stores the list entries of all created agents, each entry contains the agentId and the corresponding workspace path
- `bindings`: Located in `openclaw.json`, stores the binding rules between agents and interaction channels, determining which agent processes messages from which channel